

Week 3 Lab Assistant Preparation Guide

Technology Projects - Programming Levels on Raspberry Pi

Core idea: students use the Raspberry Pi to see how the same simple computation can be represented at different levels: assembly, C, C++, Python, and Scratch.

1. What Week 3 is about

Week 3 is about showing students the same simple computation at different levels of abstraction:

```
5 + 3 = 8
```

```
Machine code / ARM Assembly -> C -> C++ -> Python -> Scratch
```

The main teaching idea is that computers eventually execute low-level machine instructions, but programmers usually write in higher-level languages because they are easier to understand and maintain.

Students will use the Raspberry Pi as an ARM-based computer and explore how code is written, compiled, interpreted, run, and inspected.

The week is not about making students experts in assembly. It is about helping them see the connection between different programming levels.

2. Relevant repo folders/files

The Week 3 repo content is under:

```
week03/
```

Important files:

```
week03/README.md
week03/journal_programming.md
week03/00_environment.md
week03/01_assembly.md
week03/02_c.md
week03/03_cpp.md
week03/04_python.md
week03/05_scratch.md
```

Students are expected to create this folder structure:

```
week03/
|- journal/
|- asm/
|- c/
|- cpp/
|- python/
|- scratch/
```

Expected student-created files include assembly, C, C++, Python, Scratch, and journal files:

```
asm/add.s
asm/hello_arm.s
asm/add.o
asm/add
asm/add.disasm.txt
asm/hello_arm.o
asm/hello_arm
asm/hello_arm.disasm.txt

c/main.c
c/main
c/main.disasm.head.txt

cpp/main.cpp
cpp/main
cpp/main_cpp.disasm.head.txt

python/script.py

scratch/...

journal/00_setup.md
journal/01_asm_add.md
journal/02_asm_hello.md
journal/03_c.md
journal/04_cpp.md
journal/05_python.md
journal/06_scratch.md
journal/99_summary.md

Optional:
journal/07_debugging.md
```

3. What students are expected to do

- Install the programming tools on the Pi.
- Check versions of tools like gcc, g++, as, ld, and python3.
- Write a very small ARM assembly program.
- Assemble and link the assembly program.
- Use objdump to inspect the generated machine instructions.
- Write a second assembly program that prints "Hello ARM Pi!".
- Write equivalent simple programs in C, C++, and Python.
- Compare compiled languages with interpreted languages.
- Create a Scratch version of the same idea.
- Commit their work at checkpoints.
- Write short journal reflections with screenshots and explanations.

The main output they should understand is:

```
Result: 8
```

But the deeper learning is how the same logic looks different in each language.

4. What you should personally understand before the lab

A. ARM vs x86

The Raspberry Pi uses ARM architecture. ARM and x86 are different families of processor instructions. A program compiled for x86 will not automatically run as native ARM machine code, because the CPU understands a different instruction language. Students do not need deep CPU architecture knowledge, but they should understand that assembly code is architecture-specific.

```
ARM = the instruction language used by Raspberry Pi CPUs.  
x86 = the instruction language used by many Intel/AMD laptops and desktops.
```

B. Assembly, assembler, linker, executable

The assembler translates assembly into machine code inside an object file. The linker turns the object file into a runnable program.

```
as add.s -o add.o  
ld add.o -o add  
./add  
  
add.s      -> human-written assembly source  
add.o      -> object file produced by assembler  
add        -> executable file produced by linker  
./add      -> run the executable
```

C. What `_start` means

`_start` is the entry point. It tells the system where the program begins. In C and C++, students usually write `int main()`, but in raw assembly they give the lower-level entry point manually.

```
.global _start  
_start:
```

D. Registers

Registers are tiny storage spaces inside the CPU. Assembly instructions usually work directly with registers. `mov r0, #5` puts 5 into r0. `add r0, r0, #3` adds 3 and stores the result back in r0. After this, `r0 = 8`.

```
mov r0, #5  
add r0, r0, #3
```

E. System calls

A system call is how a low-level program asks the operating system to do something, such as print text or exit. The repo uses 32-bit ARM-style syscall examples with `r7`; on a 64-bit Raspberry Pi OS, this may not work exactly depending on toolchain and environment.

```
mov r0, #1          @ stdout  
ldr r1, =msg        @ buffer  
mov r2, #14         @ length  
mov r7, #4          @ syscall: write  
svc #0  
  
mov r7, #1  
svc #0
```

F. Compilation vs interpretation

C and C++ are compiled: a compiler translates source code into a machine-code executable before the program runs. Python is usually interpreted: Python code is run by the Python interpreter. Scratch is even higher-level because students build logic using visual blocks.

```
gcc main.c -o main
g++ main.cpp -o main
python3 script.py
```

Important assembly warning: the first add program may not visibly print anything. It calculates 8 in a register, but there is no print instruction. The program may run, but nothing appears on screen.

5. Common student mistakes and how to fix them

Mistake: Wrong folder

They run commands from the wrong directory.

```
pwd
ls
cd week03/asm
ls
```

Fix/explanation: Always check where you are before running the command.

Mistake: Folder does not exist

They try `cd asm` and get No such file or directory.

```
mkdir -p week03/{journal,asm,c,cpp,python,scratch}
```

Fix/explanation: Create the expected folder structure first.

Mistake: File name mismatch

They save `add.S`, `Add.s`, or `adds.s`, but the command expects `add.s`.

```
ls
mv Add.s add.s
```

Fix/explanation: Linux file names are case-sensitive.

Mistake: They expect `./add` to print something

The simple assembly program changes a register but does not print output.

```
mov r0, #5
add r0, r0, #3
```

Fix/explanation: It stores 8 inside a CPU register, but it does not print it to the screen.

Mistake: Permission error when running executable

They may see Permission denied.

```
chmod +x add
./add
```

Fix/explanation: Usually `ld` creates an executable already, but this is useful to know.

Mistake: command not found

`gcc`, `g++`, `as`, `objdump`, or `python3` may be missing.

```
sudo apt update
sudo apt install -y build-essential gcc g++ binutils python3 python3-pip gdb
```

Fix/explanation: Install the required programming tools.

Mistake: Wrong string length in assembly

If students change the message, they must update the byte length.

```
.ascii "Hello ARM Pi!\n"
mov r2, #14
```

Fix/explanation: Hello ARM Pi! plus newline is 14 bytes.

Mistake: C newline formatting error

The README may show `printf` incorrectly split across lines.

```
printf("Result: %d\n", y);
```

Fix/explanation: Use `\n` inside the string.

Mistake: C++ file compiled with gcc

Students may try `gcc main.cpp -o main`.

```
g++ main.cpp -o main
```

Fix/explanation: Use `gcc` for C and `g++` for C++.

Mistake: Python script not found

They run `python3 script.py` from the wrong folder.

```
cd week03/python
python3 script.py
```

Fix/explanation: Run it from the correct folder or use the full path.

Mistake: Confusing source, object, executable

Students may not know which file to run.

```
.s, .c, .cpp, .py = source files
.o = object file, not usually run directly
no extension, like add or main = executable
```

Fix/explanation: Python files are run by `python3` directly.

Mistake: objdump output looks scary

Students may feel lost when viewing disassembly.

```
objdump -d add > add.disasm.txt
```

Fix/explanation: They are not expected to understand every line. The goal is to notice that compiled output is more detailed than source code.

6. Commands/tools/concepts you should be ready to explain

Basic navigation

```
pwd
ls
cd week03
mkdir asm c cpp python scratch journal
```

Creating files from terminal

```
nano asm/add.s
# or
touch asm/add.s
```

Installing tools

```
sudo apt update
sudo apt install -y build-essential gcc g++ binutils python3 python3-pip gdb

# Possibly Scratch:
sudo apt install scratch
```

Checking versions

```
gcc --version
g++ --version
as --version
ld --version
python3 --version
```

Assembly build process

```
cd week03/asm
as add.s -o add.o
ld add.o -o add
./add
objdump -d add > add.disasm.txt
```

Assembly hello program

```
as hello_arm.s -o hello_arm.o
ld hello_arm.o -o hello_arm
./hello_arm
objdump -d hello_arm > hello_arm.disasm.txt
```

C compile/run

```
cd week03/c
gcc main.c -o main
./main
objdump -d main | head -n 60 > main.disasm.head.txt
```

C++ compile/run

```
cd week03/cpp
g++ main.cpp -o main
./main
objdump -d main | head -n 60 > main_cpp.disasm.head.txt
```

Python run

```
cd week03/python
python3 script.py
```

File size comparison

```
ls -lh main
# or from week03:
ls -lh c/main cpp/main
```

Git checkpoint idea

```
git status
git add .
git commit -m "setup"
git push
```

7. Physical or software setup to check before the session

- Raspberry Pis boot correctly.
- Students can log in.
- Internet works on the Pi.
- Terminal opens.
- Keyboard and mouse work.
- Display/HDMI works if using monitors.
- SSH/VNC works if you plan to support remotely.
- sudo apt update works.
- Toolchain installation works.

- Scratch is installed or installable.
- GitHub Classroom repo access works.
- Students can push commits.
- Enough disk space exists.

Useful checks:

```
uname -m
cat /etc/os-release
gcc --version
g++ --version
as --version
ld --version
python3 --version
git --version

# Possible architecture outputs:
armv7l -> 32-bit ARM
aarch64 -> 64-bit ARM
```

8. Simple explanations you can give to beginners

What is Machine code?

Machine code is the real instruction format the CPU understands. Humans usually do not write it directly because it is hard to read.

What is Assembly?

Assembly is a human-readable version of machine code. It is still very close to the CPU.

What is C?

C is a compiled language. It is higher-level than assembly, but still close enough to show how memory and machine instructions work.

What is C++?

C++ is also compiled, but it has more features than C, like classes and a large standard library.

What is Python?

Python is a high-level interpreted language. It is easier to write, but the Python interpreter does more work behind the scenes.

What is Scratch?

Scratch is visual programming. Instead of typing syntax, you connect blocks to create logic.

What is Compiler?

A compiler translates source code into a program the computer can run.

What is Interpreter?

An interpreter reads and runs code through another program, like Python running .py files.

What is objdump?

objdump lets us look inside a compiled program and see the low-level instructions.

Why compare languages?

To see that the same idea can be expressed at different levels. Lower-level code is closer to the hardware. Higher-level code is easier for humans.

9. Suggested lab flow for you as assistant

Step: Start with the big idea

Today we are doing the same small calculation in different programming levels: $5 + 3$.

Scratch/Python = easiest to read

C/C++ = compiled into machine code

Assembly = very close to what the CPU executes

Step: Set up folders

```
mkdir -p week03/{journal,asm,c,cpp,python,scratch}
cd week03
```

Step: Environment setup

```
sudo apt update
sudo apt install -y build-essential gcc g++ binutils python3 python3-pip gdb
```

Step: Assembly add

```
.global _start
_start:
    mov r0, #5
    add r0, r0, #3

# Build:
cd asm
as add.s -o add.o
ld add.o -o add
./add
objdump -d add > add.disasm.txt

# Warning: this may not print anything. That is expected.
```

Step: Assembly hello

```
as hello_arm.s -o hello_arm.o
ld hello_arm.o -o hello_arm
./hello_arm
```

Step: C

```
#include <stdio.h>

int main() {
    int x = 5;
    int y = x + 3;
    printf("Result: %d\n", y);
    return 0;
}

# Compile/run:
gcc main.c -o main
./main
```

Step: C++

```
#include <iostream>
using namespace std;

int main() {
    int x = 5;
    int y = x + 3;
    cout << "Result: " << y << endl;
    return 0;
}

# Compile/run:
g++ main.cpp -o main
./main
```

Step: Python

```
x = 5
y = x + 3
print("Result:", y)

# Run:
python3 script.py
```

Step: Scratch

```
set x to 5
change x by 3
say x
```

Step: Journal and commits

```
screenshots
version outputs
disassembly snippets
short reflections
commits
```

10. Common questions students may ask

Why does assembly not print 8?

The code only changes a register. It does not call the operating system to print anything. The CPU knows the answer, but the screen does not.

What is r0?

r0 is a CPU register. Think of it as a tiny variable inside the CPU.

Why do we need ld after as?

as assembles source into an object file. ld links the object file into an executable.

Why do we use ./main?

./main means run the file called main in the current folder. Linux does not automatically run programs from the current folder unless you specify the path.

Why does C++ produce more complex disassembly than C?

C++ often brings in more runtime/library code, especially when using iostream, cout, and endl.

Why is Python easier?

Python hides many low-level details. print("Result:", y) does a lot behind the scenes, but the programmer does not manually manage registers or system calls.

What is the point of Scratch?

Scratch shows programming logic without syntax problems. It helps students understand the idea before worrying about semicolons, compilers, and commands.

11. Important missing/incomplete/unclear parts in the repo

1. Possible issue with 32-bit vs 64-bit ARM assembly

The assembly examples use 32-bit ARM style registers and syscall convention. This is typical for 32-bit ARM Linux. But many Raspberry Pi OS installs are now 64-bit, where assembly uses registers like x0, x1, x2, and x8, and syscall numbers differ.

```
r0
r1
r2
r7
svc #0

# Check architecture:
uname -m
```

If output is armv7l, the repo's 32-bit-style assembly is more likely to work. If output is aarch64, staff may need to clarify whether students should use 32-bit Raspberry Pi OS, use 32-bit compatibility, or receive AArch64 assembly examples. This is the biggest technical risk for Week 3.

2. README has a formatting error in the C example

```
printf("Result: %d\n", y);
```

3. Scratch installation may vary

The README suggests installing scratch with apt, but on some Raspberry Pi OS versions, Scratch may not install using exactly scratch, or it may already be available from the desktop menu. Staff should check this before the lab.

```
sudo apt install scratch
```

4. The assembly add program has no proper exit syscall

The minimal add program does not explicitly exit. A more complete assembly program usually ends with an exit syscall. Students might see unusual behavior depending on environment. Staff may need to clarify whether the minimal example is only for disassembly inspection, not a proper complete program.

5. Journal expectations are spread across multiple files

The repo includes several guide files, so students may not know whether to follow README.md or journal_programming.md.

Use journal_programming.md as the full checkpoint guide. Use the smaller files as separated checkpoint instructions.

12. Short checklist before the lab

- ☐ Pi boots and students can log in
- ☐ Internet works
- ☐ Terminal works
- ☐ GitHub Classroom repo is accessible
- ☐ week03 folder structure is clear
- ☐ gcc, g++, as, ld, objdump, python3 are installed
- ☐ Scratch is installed or available
- ☐ You know whether the Pi OS is 32-bit or 64-bit
- ☐ You have tested the assembly examples on the actual lab Pis
- ☐ You know how to explain registers r0, r1, r2, r7
- ☐ You know how to explain assembler vs linker
- ☐ You know how to explain compiler vs interpreter
- ☐ You are ready to fix C newline formatting issue
- ☐ You are ready to help with wrong-folder and file-name mistakes
- ☐ You remind students to take screenshots and commit after checkpoints

13. One-sentence beginner explanation for the whole week

This week shows how the same simple calculation can be written in many ways, from very low-level CPU instructions to high-level visual programming, so students understand what programming languages hide or reveal.